

Adaptive Multimodal Fusion for Social Media Content Classification using PySpark

S. M. Saidur Rahman (22101451)
Shayonto Rayhan (22101651)

CSE761 Project Report

Problem & Motivation

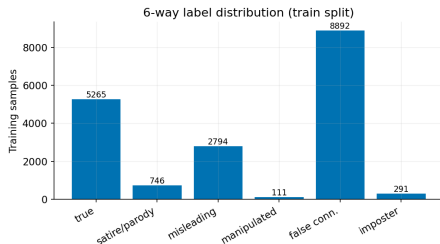
- Misinformation on social platforms is often **multimodal**: a real image paired with a misleading caption, or an old photo reused alongside a false claim about a current event.
- Neither the text nor the image is false on its own – only the *pairing* is. A text-only or image-only detector cannot catch this, since the misleading signal exists only in how the two relate.
- Real posts don't always arrive with both modalities intact: images go missing, links rot, uploads fail.
- **Big-data angle**: millions of posts, each needing an image download, an image embedding, and a text embedding – a single-machine pipeline bottlenecks at this scale. We build ours on Apache Spark, which parallelizes cleaning, feature extraction, and GPU-batched embedding across workers.
- **Core question**: since we can't always trust both modalities equally, can a model that *adapts* its trust per modality, per sample, beat one that always weights both the same way?

Dataset

- Fakeddit multimodal_only_samples, 6-way label: *true*, *satire/parody*, *misleading*, *manipulated*, *false connection*, *imposter*.

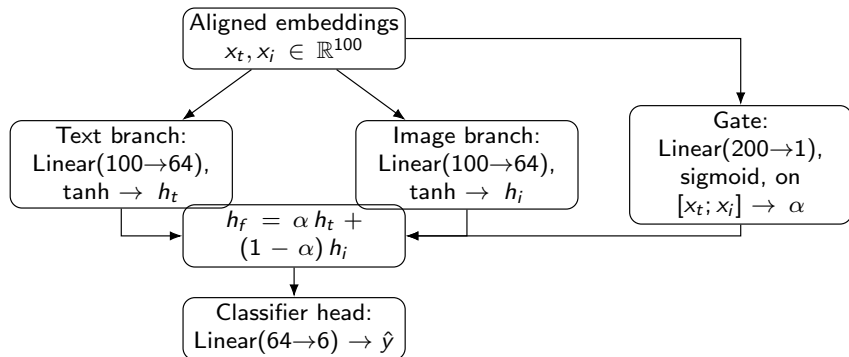
- Heavily imbalanced: *false connection* = 49.1% of train; *manipulated* (0.6%), *imposter* (1.6%) are rare.

⇒ Majority-class guessing scores $\sim 49\%$ – numbers should be judged against that floor.



- Stratified sample, target 30,000/5,000/5,000 train/val/test – preserves class proportions from the full $\sim 1\text{M}$ -row release.
- Sampling (not the full release) keeps image-download + GPU embedding tractable at course scale; the Spark pipeline itself is architected to scale unchanged.
- 2019-era Reddit image links suffer heavy rot $\rightarrow$ only 60.3% of sampled rows have a resolvable image.
- Final usable split: **18,099 / 1,401 / 1,443** (train/val/test).
- Evaluated under three test conditions: full (both modalities), image-missing, text-missing.

Adaptive-Joint Architecture



Our main contribution: α is trained end-to-end with the rest of the network,
unlike adaptive-fused.

Models Compared

- **static-concat** (baseline): 200-dim concat $\rightarrow$ linear LogisticRegression. Trained on clean data.
- **static-concat-aug**: same architecture, trained on modality-dropout *augmented* data.
- **adaptive-fused**: α from two frozen, separately pre-trained unimodal MLPs $\rightarrow \alpha$ -weighted concat $\rightarrow$ LogisticRegression.
- **adaptive-routed**: inference-time rule – defer to the matching frozen unimodal MLP when a modality is missing.
- **rf-concat**: same 200-dim input, nonlinear RandomForestClassifier (Spark's GBTCClassifier doesn't support multiclass).
- **adaptive-joint** (final): one PyTorch network, text + image branches + gate, trained end-to-end with a single loss.

Fusion: Two Ways to Use α

adaptive-fused – weights the *raw* embeddings, α comes from frozen, separately-trained models:

$$f_{\text{fused}} = [\alpha x_t ; (1 - \alpha) x_i ; \alpha]$$

adaptive-joint – weights learned branch representations h_t, h_i ; α trained jointly with everything else:

$$h_f = \alpha h_t + (1 - \alpha) h_i$$

⇒ In adaptive-joint, α gets **direct gradient feedback** on whether its weighting choice helped the prediction. In adaptive-fused, it doesn't.

Experimental Setup

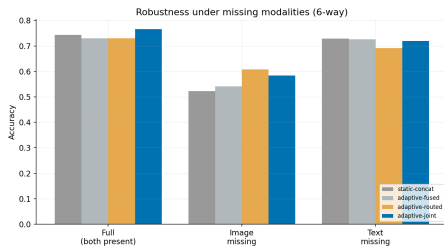
Text encoder	Word2Vec, 100-dim (frozen)
Image encoder	ResNet-18, ImageNet-pretrained, frozen, 512→100 via PCA
Modality dropout	15% image-dropped + 15% text-dropped
Non-joint classifiers	Spark MLlib LogisticRegression
Joint network	100→64 (text) + 100→64 (image) + gate + head Adam, $lr = 10^{-3}$, 30 epochs, batch 256
Hardware	Kaggle, 2× NVIDIA T4, Spark local[*]
Test conditions	Full, image-missing, text-missing

Results

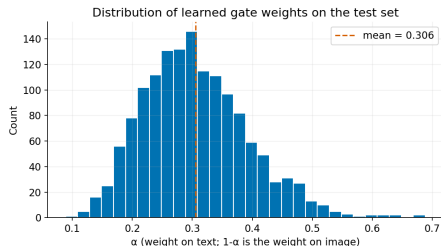
Model	Full		Image missing		Text missing	
	Acc.	F1	Acc.	F1	Acc.	F1
text-only	0.428	0.448	—	—	—	—
image-only	0.514	0.530	—	—	—	—
static-concat (baseline)	0.562	0.569	0.130	0.169	0.529	0.537
static-concat-aug	0.565	0.571	0.148	0.191	0.526	0.534
rf-concat	0.547	0.549	0.558	0.497	0.474	0.362
adaptive-fused	0.518	0.514	0.223	0.275	0.548	0.554
adaptive-routed	0.518	0.514	0.428	0.448	0.514	0.530
adaptive-joint (final)	0.651	0.657	0.373	0.424	0.575	0.563

- **adaptive-joint beats the baseline in every condition:** full (+8.9 pts), image-missing (+24.3 pts), text-missing (+4.6 pts).
- ⇒ But adaptive-joint is not the top score everywhere: on image-missing, adaptive-routed (0.428) is still higher than adaptive-joint (0.373) – routing to a dedicated unimodal model wins that specific case.

Results: Robustness & Gate Behavior



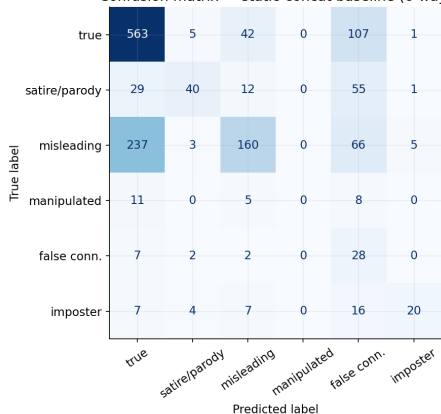
Accuracy across all three test conditions.



Distribution of the learned gate weight α on the test set.

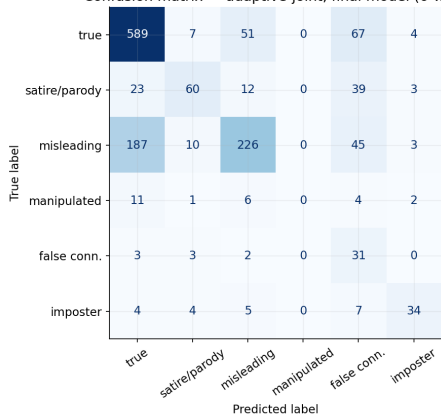
Confusion Matrices: Baseline vs. Final Model

Confusion matrix — static-concat baseline (6-way)



static-concat

Confusion matrix — adaptive-joint, final model (6-way)



adaptive-joint

Why naive gating fails. adaptive-fused's α comes from two independently trained, frozen confidence models – never optimized against the downstream loss. A linear classifier can't correct a per-dimension multiplicative distortion, so static-concat beats every non-joint adaptive variant on clean data.

Why joint training fixes it. Training α and the classifier together gives α direct gradient feedback – the only change that lets an adaptive design beat the baseline, in all three conditions. Learned α : mean 0.306, std 0.088.

Why static-concat is naturally robust. A linear classifier computes each modality's contribution independently ($w_{\text{text}}x_{\text{text}} + w_{\text{image}}x_{\text{image}}$); zeroing one term needs no retraining – the robustness is architectural, not from training exposure.

Limitations & Future Work

- **Scale:** run on a stratified sample, not the full $\sim 1\text{M}$ -row release; pipeline architected to scale unchanged.
- **Class imbalance:** *manipulated content* (111 examples) and *imposter content* (291) too rare for reliable per-class conclusions.
- **Partial (not missing) information** not separately measured – only fully-missing-modality tested.
- **Distributed joint training:** currently single-process PyTorch; Spark's TorchDistributor could extend this to multi-node.
- **Routing + joint hybrid:** combine adaptive-routed's image-missing floor with adaptive-joint's overall ceiling.

Conclusion

- Built a distributed PySpark pipeline for fine-grained (6-way) multimodal fake news classification.
- Naive confidence-gated fusion does **not** outperform simple concatenation – traced to the gate never being optimized against the downstream loss.
- Joint, end-to-end training resolves this: the only fusion variant to beat the concatenation baseline across **all** evaluated conditions.

- K. Nakamura, S. Levy, W. Y. Wang, “r/Fakeddit: A New Multimodal Benchmark Dataset for Fine-grained Fake News Detection,” LREC, 2020.
- J. Arevalo et al., “Gated Multimodal Units for Information Fusion,” ICLR Workshop, 2017.
- K. He et al., “Deep Residual Learning for Image Recognition,” CVPR, 2016.
- Apache Spark MLlib documentation.

Thank you – Questions?